

LAPORAN PRATIKUM
STRUKTUR DATA
“Laporan Praktikum Pekan 6”

Disusun Oleh:

Zaki Misucha Andefi

2511533006

Dosen Pengampu : Dr. Wahyudi, S.T, M.T.

Asisten Praktikum : Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum Struktur Data Pekan 6 ini dapat diselesaikan dengan baik dan tepat waktu. Laporan ini dibuat sebagai salah satu bentuk tugas praktikum pada mata kuliah Struktur Data yang bertujuan untuk memahami konsep serta implementasi struktur data Double Linked List menggunakan bahasa pemrograman Java.

Pada praktikum kali ini, penulis mempelajari berbagai operasi dasar pada Double Linked List (DLL), seperti penambahan node, penghapusan node, serta penelusuran data secara maju dan mundur. Dengan adanya pointer next dan prev pada setiap node, Double Linked List memiliki kemampuan traversal dua arah yang lebih fleksibel dibandingkan Single Linked List.

Melalui pembuatan beberapa program praktikum, penulis dapat memahami bagaimana proses manipulasi node dilakukan pada struktur data DLL, baik ketika menambahkan node di awal, akhir, maupun posisi tertentu, serta saat menghapus node dari linked list.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan baik dari segi isi maupun penulisan. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan laporan di masa mendatang.

Akhir kata, semoga laporan ini dapat memberikan manfaat dan menambah wawasan pembaca mengenai implementasi struktur data Double Linked List dalam pemrograman Java.

Padang, April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN	2
2.1 Langkah Kerja.....	2
2.2 Program Iterasi Queue	2
2.3 Program Queue Array	3
2.4 Program Queue Array Driver.....	3
2.5 Program Queue Linked List.....	5
2.5 Program Reverse Data	5
BAB III KESIMPULAN.....	6
DAFTAR PUSTAKA	7

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan bagian penting dalam ilmu pemrograman karena digunakan untuk mengatur, menyimpan, dan mengelola data agar proses pengolahan informasi dapat dilakukan secara efisien. Salah satu struktur data dinamis yang sering digunakan adalah Linked List. Linked List memiliki beberapa jenis, salah satunya yaitu Double Linked List (DLL).

Double Linked List adalah struktur data yang terdiri dari kumpulan node, di mana setiap node memiliki data, pointer ke node berikutnya (next), dan pointer ke node sebelumnya (prev). Dengan adanya dua pointer tersebut, proses traversal dapat dilakukan dari depan ke belakang maupun dari belakang ke depan.

Double Linked List memiliki kelebihan dibandingkan Single Linked List karena lebih fleksibel dalam proses manipulasi data, terutama pada operasi penghapusan dan penelusuran node. Struktur ini banyak digunakan pada implementasi aplikasi seperti navigasi browser, playlist musik, undo-redo editor, dan sistem antrian tertentu.

Dalam praktikum ini dilakukan implementasi berbagai operasi dasar Double Linked List menggunakan bahasa Java, seperti penambahan node, penghapusan node, dan traversal data. Dengan praktikum ini diharapkan mahasiswa dapat memahami konsep kerja Double Linked List serta penerapannya dalam pemrograman.

1.2 Tujuan

Memahami konsep Double Linked List serta mengimplementasikan operasi-operasi dasar seperti insert, delete, dan traversal menggunakan bahasa pemrograman Java.

1.3 Manfaat

Meningkatkan pemahaman mengenai struktur data Double Linked List, melatih logika pemrograman berbasis pointer dua arah, serta memberikan pengalaman dalam melakukan manipulasi data secara dinamis menggunakan Java..

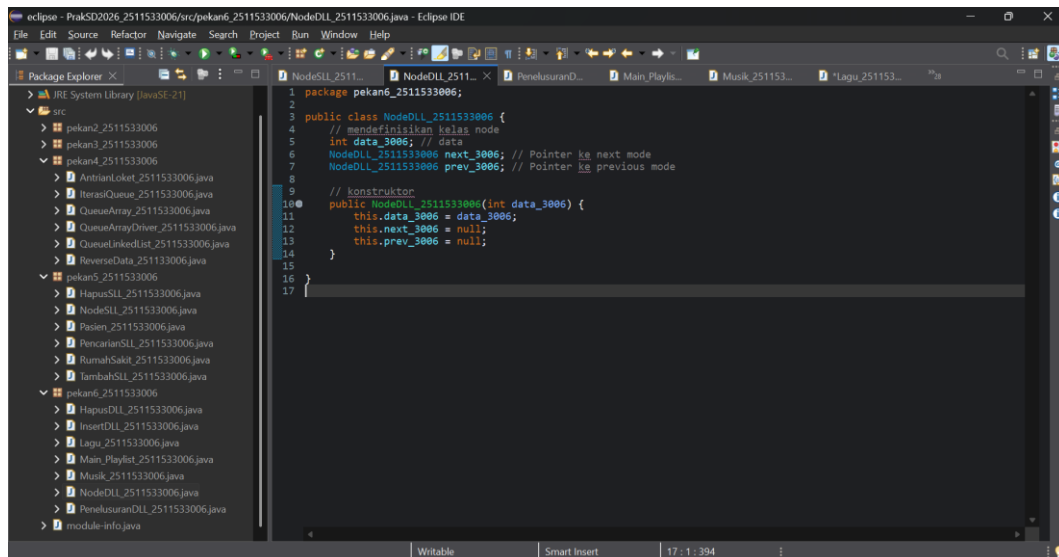
BAB II

PEMBAHASAN

2.1 Langkah Kerja

1. Membuka IDE (Eclipse).
2. Membuat package baru bernama pekan4_2511533006.
3. Menambahkan beberapa class:
 - NodeDLL_2511533006
 - InsertDLL_2511533006
 - HapusDLL_2511533006
 - PenelusuranDLL_2511533006
4. Menjalankan program satu per satu dan mengamati hasilnya.

2.2 Program Node DLL

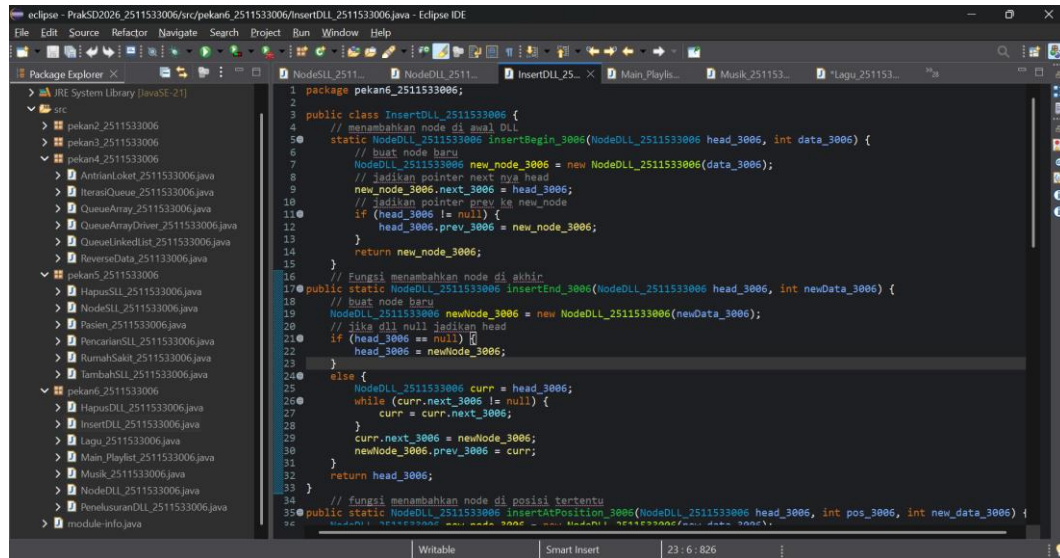


Class NodeDLL_2511533006 digunakan sebagai struktur dasar dari Double Linked List. Pada class ini terdapat atribut data untuk menyimpan nilai integer, pointer next untuk menunjuk node berikutnya, dan pointer prev untuk menunjuk node sebelumnya.

Analisis: Program ini berfungsi sebagai blueprint atau rancangan dasar dari setiap node pada Double Linked List. Variabel data_3006 digunakan untuk menyimpan data integer, sedangkan next_3006 menunjuk ke node berikutnya dan prev_3006 menunjuk ke node sebelumnya. Konstruktor pada class digunakan untuk menginisialisasi nilai data dan

mengatur pointer next serta prev menjadi null. Class ini menjadi dasar utama dalam implementasi operasi-operasi lain pada Double Linked List.

2.3 Program Insert DLL

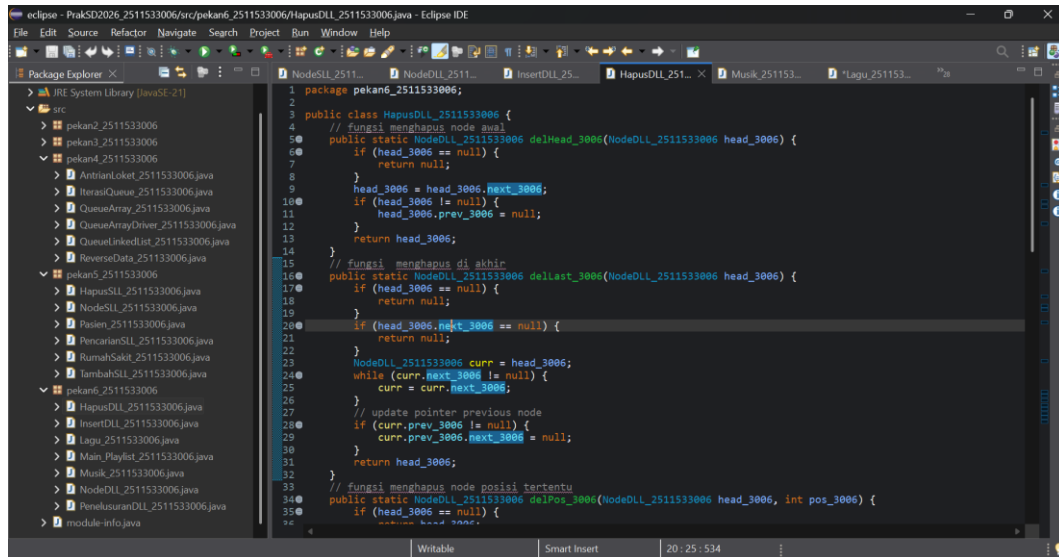


```
1 package pekan6_2511533006;
2
3 public class InsertDLL_2511533006 {
4     // menambahkan node di awal DLL
5     static NodeDLL_2511533006 insertBegin_3006(NodeDLL_2511533006 head_3006, int data_3006) {
6         // buat node baru
7         NodeDLL_2511533006 new_node_3006 = new NodeDLL_2511533006(data_3006);
8         // jadikan pointer next nya head
9         new_node_3006.next_3006 = head_3006;
10        // jadikan pointer prev ke new_node
11        if (head_3006 != null) {
12            head_3006.prev_3006 = new_node_3006;
13        }
14        return new_node_3006;
15    }
16    // Fungsi menambahkan node di akhir
17    public static NodeDLL_2511533006 insertEnd_3006(NodeDLL_2511533006 head_3006, int newData_3006) {
18        // buat node baru
19        NodeDLL_2511533006 new_node_3006 = new NodeDLL_2511533006(newData_3006);
20        // jika di null jadikan head
21        if (head_3006 == null) {
22            head_3006 = new_node_3006;
23        }
24        else {
25            NodeDLL_2511533006 curr = head_3006;
26            while (curr.next_3006 != null) {
27                curr = curr.next_3006;
28            }
29            curr.next_3006 = new_node_3006;
30            new_node_3006.prev_3006 = curr;
31        }
32        return head_3006;
33    }
34    // fungsi menambahkan node di posisi tertentu
35    public static NodeDLL_2511533006 insertAtPosition_3006(NodeDLL_2511533006 head_3006, int pos_3006, int new_data_3006) {
36        // ...
37    }
38 }
```

Class InsertDLL_2511533006 mengimplementasikan operasi penambahan node pada Double Linked List, yaitu penambahan node di awal, di akhir, dan di posisi tertentu.

Analisis: Program ini mendemonstrasikan beberapa metode penambahan node pada Double Linked List. Metode insertBegin_3006 digunakan untuk menambahkan node di awal list dengan cara menjadikan node baru sebagai head baru dan menghubungkan pointer prev head lama ke node baru. Metode insertEnd_3006 digunakan untuk menambahkan node di akhir list dengan menelusuri node hingga akhir kemudian menghubungkannya dengan node baru. Sedangkan metode insertAtPosition_3006 digunakan untuk menambahkan node pada posisi tertentu dengan mengatur hubungan pointer next dan prev antar node. Output program menunjukkan bahwa data berhasil ditambahkan di awal, akhir, dan posisi tertentu secara benar.

2.4 Program Hapus DLL

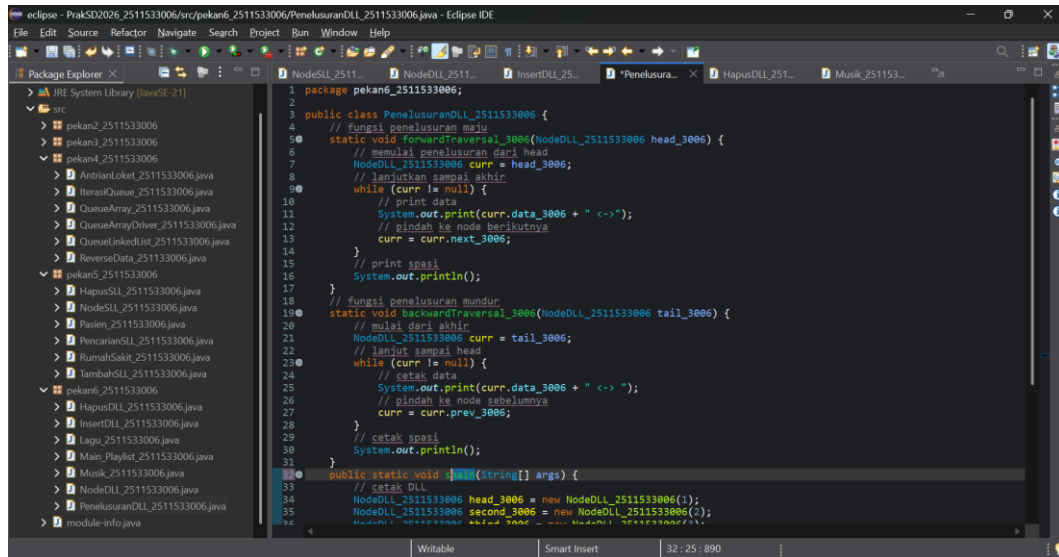


```
1 package pekan6_2511533006;
2
3 public class HapusDLL_2511533006 {
4     // fungsi menghapus node awal
5     public static NodeDLL_2511533006 delHead_3006(NodeDLL_2511533006 head_3006) {
6         if (head_3006 == null) {
7             return null;
8         }
9         head_3006 = head_3006.next_3006;
10        if (head_3006 != null) {
11            head_3006.prev_3006 = null;
12        }
13        return head_3006;
14    }
15    // fungsi menghapus di akhir
16    public static NodeDLL_2511533006 delLast_3006(NodeDLL_2511533006 head_3006) {
17        if (head_3006 == null) {
18            return null;
19        }
20        if (head_3006.next_3006 == null) {
21            return null;
22        }
23        NodeDLL_2511533006 curr = head_3006;
24        while (curr.next_3006 != null) {
25            curr = curr.next_3006;
26        }
27        // update pointer previous node
28        if (curr.prev_3006 != null) {
29            curr.prev_3006.next_3006 = null;
30        }
31        return head_3006;
32    }
33    // fungsi menghapus node posisi tertentu
34    public static NodeDLL_2511533006 delPos_3006(NodeDLL_2511533006 head_3006, int pos_3006) {
35        if (head_3006 == null) {
36            return null;
37        }
38    }
```

Class HapusDLL_2511533006 digunakan untuk mengimplementasikan operasi penghapusan node pada Double Linked List, yaitu penghapusan node awal, node akhir, dan node pada posisi tertentu.

Analisis: Program ini menunjukkan proses penghapusan node pada Double Linked List. Metode delHead_3006 menghapus node pertama dengan memindahkan head ke node berikutnya dan mengatur prev menjadi null. Metode delLast_3006 digunakan untuk menghapus node terakhir dengan menelusuri list hingga akhir lalu memutus hubungan node sebelumnya. Sedangkan metode delPos_3006 digunakan untuk menghapus node pada posisi tertentu dengan menghubungkan node sebelumnya langsung ke node setelahnya. Program ini juga memperlihatkan pentingnya pengaturan pointer prev dan next agar struktur linked list tetap terhubung dengan baik setelah proses penghapusan dilakukan.

2.5 Program Penelusuran DLL



```
1 package pekan6_2511533006;
2
3 public class PenelusuranDLL_2511533006 {
4     // fungsi penelusuran maju
5     static void forwardTraversal_3006(NodeDLL_2511533006 head_3006) {
6         // memulai penelusuran dari head
7         NodeDLL_2511533006 curr = head_3006;
8         // lanjutkan sampai akhir
9         while (curr != null) {
10            // print data
11            System.out.print(curr.data_3006 + " <-> ");
12            // pindah ke node berikutnya
13            curr = curr.next_3006;
14        }
15        // print spasi
16        System.out.println();
17    }
18    // fungsi penelusuran mundur
19    static void backwardTraversal_3006(NodeDLL_2511533006 tail_3006) {
20        // mulai dari akhir
21        NodeDLL_2511533006 curr = tail_3006;
22        // lanjut sampai head
23        while (curr != null) {
24            // cetak data
25            System.out.print(curr.data_3006 + " <-> ");
26            // pindah ke node sebelumnya
27            curr = curr.prev_3006;
28        }
29        // cetak spasi
30        System.out.println();
31    }
32    public static void main(String[] args) {
33        // cetak DLL
34        NodeDLL_2511533006 head_3006 = new NodeDLL_2511533006(1);
35        NodeDLL_2511533006 second_3006 = new NodeDLL_2511533006(2);
36        head_3006.next_3006 = second_3006;
37        second_3006.prev_3006 = head_3006;
38    }
39 }
```

Class `PenelusuranDLL_2511533006` digunakan untuk melakukan traversal atau penelusuran data pada Double Linked List secara maju dan mundur.

Analisis: Program ini mendemonstrasikan kelebihan utama Double Linked List yaitu kemampuan traversal dua arah. Metode `forwardTraversal_3006` digunakan untuk menelusuri node dari head menuju tail menggunakan pointer `next`. Sedangkan metode `backwardTraversal_3006` digunakan untuk menelusuri node dari tail menuju head menggunakan pointer `prev`. Dengan adanya dua arah traversal ini, Double Linked List menjadi lebih fleksibel dibandingkan Single Linked List dalam pengolahan data tertentu.

BAB III

KESIMPULAN

Double Linked List (DLL) merupakan salah satu struktur data dinamis yang sangat penting dalam pemrograman karena memungkinkan proses penyimpanan dan pengelolaan data secara fleksibel. Berbeda dengan Single Linked List yang hanya memiliki satu pointer, DLL memiliki dua pointer yaitu next dan prev sehingga traversal dapat dilakukan dua arah.

Melalui praktikum ini dapat dipahami bahwa setiap node pada Double Linked List terdiri dari data, pointer next, dan pointer prev. Dengan struktur tersebut, proses manipulasi data seperti penambahan, penghapusan, dan penelusuran node dapat dilakukan dengan lebih mudah dan efisien.

Operasi penambahan node dapat dilakukan di awal, akhir, maupun posisi tertentu dengan cara mengatur hubungan antar pointer pada node-node yang terlibat. Selain itu, operasi penghapusan node juga memerlukan pengaturan pointer next dan prev agar linked list tetap tersusun dengan benar setelah node dihapus.

Traversal pada Double Linked List dapat dilakukan secara maju maupun mundur. Kemampuan traversal dua arah ini menjadi salah satu kelebihan utama DLL dibandingkan Single Linked List karena memberikan fleksibilitas lebih tinggi dalam pengolahan data.

Dengan memahami implementasi Double Linked List menggunakan Java, mahasiswa diharapkan mampu menerapkan konsep struktur data ini pada pengembangan program yang lebih kompleks dan meningkatkan kemampuan dalam pengelolaan data secara efisien.

DAFTAR PUSTAKA

- [1] H. M. Deitel dan P. J. Deitel, *Java: How to Program*, 11th ed. New Jersey, NJ: Pearson, 2017.

Tugas Struktur Data

1. Lagu_2511533006

```
package pekan6_2511533006;

public class Lagu_2511533006 {
    private String judul_3006;
    private String penyanyi_3006;
    public Lagu_2511533006 next_3006;
    public Lagu_2511533006 prev_3006;

    // Method Constructor
    public Lagu_2511533006(String judul, String penyanyi) {
        this.judul_3006 = judul;
        this.penyanyi_3006 = penyanyi;
        this.next_3006 = null;
        this.prev_3006 = null;
    }

    // Method Getter
    public String getJudul_3006() {
        return judul_3006;
    }

    public String getPenyanyi_3006() {
        return penyanyi_3006;
    }

    // Method Setter
    public void setJudul_3006(String judul) {
        this.judul_3006 = judul;
    }

    public void setPenyanyi_3006(String penyanyi) {
        this.penyanyi_3006 = penyanyi;
    }
}
```

2. Musik_2511533006

```
package pekan6_2511533006;

public class Musik_2511533006 {
    private Lagu_2511533006 head_3006;
    private Lagu_2511533006 tail_3006;

    public Musik_2511533006() {
        this.head_3006 = null;
        this.tail_3006 = null;
    }

    // 1. Menambah lagu baru di AKHIR playlist
    public void tambahLagu_3006(String judul, String penyanyi) {
```

```

        Lagu_2511533006 laguBaru = new Lagu_2511533006(judul,
        penyanyi);

        if (head_3006 == null) {
            head_3006 = laguBaru;
            tail_3006 = laguBaru;
        } else {
            tail_3006.next_3006 = laguBaru;
            laguBaru.prev_3006 = tail_3006;
            tail_3006 = laguBaru;
        }
        System.out.println("Lagu berhasil ditambahkan!\n");
    }

    // 2. Menghapus lagu pertama (head)
    public void hapusLaguAwal_3006() {
        if (head_3006 == null) {
            System.out.println("Playlist kosong, tidak ada yang bisa
            dihapus.\n");
            return;
        }

        if (head_3006 == tail_3006) {
            head_3006 = null;
            tail_3006 = null;
        } else {
            head_3006 = head_3006.next_3006;
            head_3006.prev_3006 = null;
        }
        System.out.println("Lagu pertama berhasil dihapus!\n");
    }

    // 3. Menampilkan semua lagu dari awal ke akhir
    public void tampilMaju_3006() {
        if (head_3006 == null) {
            System.out.println("Playlist masih kosong.\n");
            return;
        }

        System.out.println("--- Playlist (Maju) ---");
        Lagu_2511533006 current = head_3006;
        int no = 1;
        while (current != null) {
            System.out.println(no + ". " + current.getJudul_3006() +
            " - " + current.getPenyanyi_3006());
            current = current.next_3006;
            no++;
        }
        System.out.println();
    }

    // 4. Menampilkan semua lagu dari akhir ke awal
    public void tampilMundur_3006() {
        if (tail_3006 == null) {
            System.out.println("Playlist masih kosong.\n");
            return;
        }
    }

```

```

        System.out.println("--- Playlist (Mundur) ---");
        Lagu_2511533006 current = tail_3006;
        int no = 1;
        while (current != null) {
            System.out.println(no + ". " + current.getJudul_3006() +
" - " + current.getPenyanyi_3006());
            current = current.prev_3006;
            no++;
        }
        System.out.println();
    }

    // 5. Mencari lagu berdasarkan judul
    public void cariLagu_3006(String judulCari) {
        if (head_3006 == null) {
            System.out.println("Playlist masih kosong.\n");
            return;
        }

        Lagu_2511533006 current = head_3006;
        boolean ditemukan = false;

        System.out.println("--- Hasil Pencarian ---");
        while (current != null) {
            if
(current.getJudul_3006().toLowerCase().contains(judulCari.toLowerCase())) {
                System.out.println("Ditemukan: " +
current.getJudul_3006() + " - " + current.getPenyanyi_3006());
                ditemukan = true;
            }
            current = current.next_3006;
        }

        if (!ditemukan) {
            System.out.println("Lagu tidak ditemukan.");
        }
        System.out.println();
    }
}

```

3. Main_Playlist_2511533006

```

package pekan6_2511533006;
import java.util.Scanner;

public class Main_Playlist_2511533006 {
    public static void main(String[] args) {
        // Variabel scanner dan playlist menggunakan akhiran NIM
        Scanner scanner_3006 = new Scanner(System.in);
        Musik_2511533006 playlist_3006 = new Musik_2511533006();
        int pilihan_3006;

        do {

```

```

32] // Tampilan menu dengan NIM lengkap sesuai contoh [cite:

System.out.println("Playlist Musik NIM: 2511533006");
System.out.println("1. Tambah Lagu");
System.out.println("2. Hapus Lagu Pertama");
System.out.println("3. Lihat Playlist (Maju)");
System.out.println("4. Lihat Playlist (Mundur)");
System.out.println("5. Cari Lagu");
System.out.println("6. Keluar");
System.out.print("Pilihan: ");

pilihan_3006 = scanner_3006.nextInt();
scanner_3006.nextLine(); // Membersihkan buffer

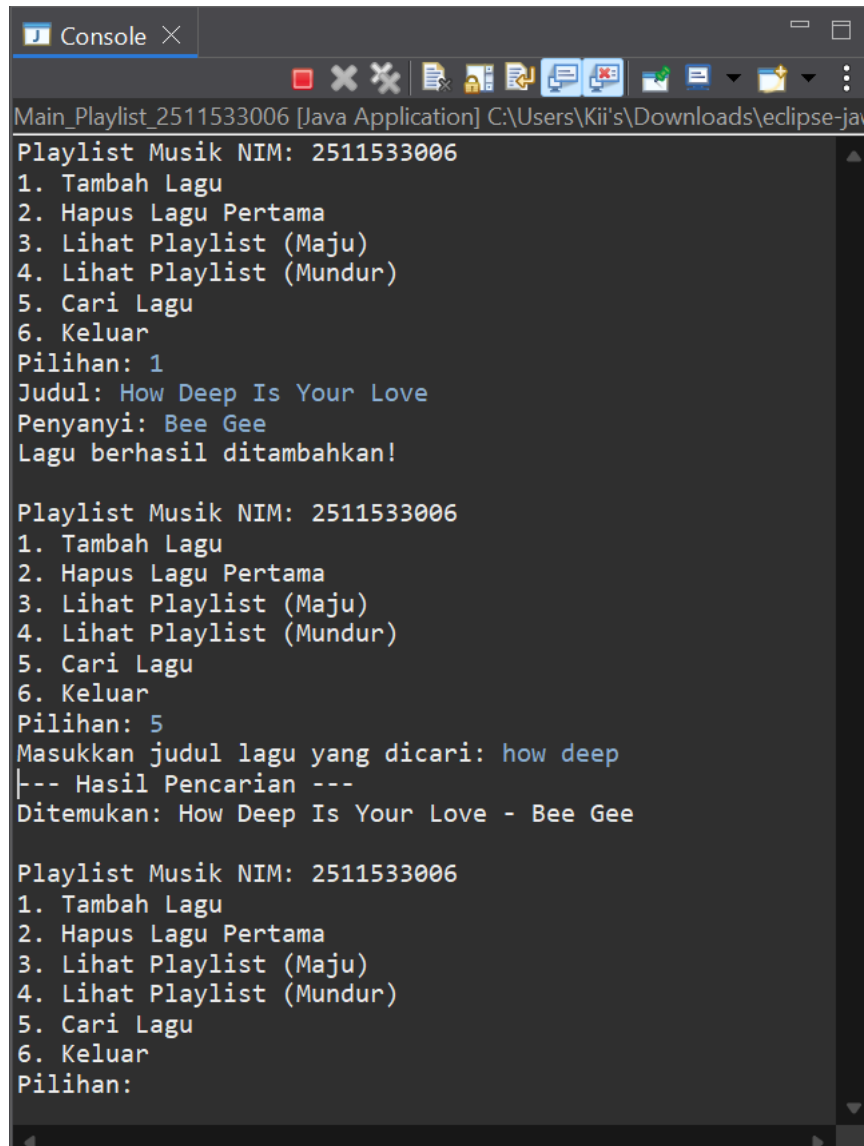
switch (pilihan_3006) {
    case 1:
        System.out.print("Judul: ");
        String judul = scanner_3006.nextLine();
        System.out.print("Penyanyi: ");
        String penyanyi = scanner_3006.nextLine();
        playlist_3006.tambahLagu_3006(judul, penyanyi);
        break;
    case 2:
        playlist_3006.hapusLaguAwal_3006();
        break;
    case 3:
        playlist_3006.tampilMaju_3006();
        break;
    case 4:
        playlist_3006.tampilMundur_3006();
        break;
    case 5:
        System.out.print("Masukkan judul lagu yang
dicari: ");

        String judulCari = scanner_3006.nextLine();
        playlist_3006.cariLagu_3006(judulCari);
        break;
    case 6:
        System.out.println("Keluar dari program
playlist.");
        break;
    default:
        System.out.println("Pilihan tidak valid!\n");
}
} while (pilihan_3006 != 6);

scanner_3006.close();
}
}

```

4. Output



```
Console X
Main_Playlist_2511533006 [Java Application] C:\Users\Kii's\Downloads\eclipse-ja
Playlist Musik NIM: 2511533006
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: How Deep Is Your Love
Penyanyi: Bee Gee
Lagu berhasil ditambahkan!

Playlist Musik NIM: 2511533006
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu yang dicari: how deep
|--- Hasil Pencarian ---
Ditemukan: How Deep Is Your Love - Bee Gee

Playlist Musik NIM: 2511533006
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan:
```